



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



DESIGN AND ANALYSIS OF ALGORITHMS

QUESTION BANK





S.No	Short Answer Questions	Page no
1	What is meant by Dynamic Programming with suitable example.	
2	What are the key properties for Dynamic Programming?	
3	What are the Applications of Dynamic Programming with detail.	

S.No	Long Answer Questions	Page no
1	Difference between Divide & Conquer (D&C) and Dynamic Programming (DP)	
2	Define Dynamic Programming. Explain the elements of Dynamic Programming and write its general control abstraction algorithm.	
3	Consider a 0/1 Knapsack Problem with $n = 4$ items having weights $w_1 = 1$, $w_2 = 2$, $w_3 = 2$, $w_4 = 3$ and corresponding profits $p_1 = 1$, $p_2 = 3$, $p_3 = 4$, $p_4 = 5$. If the knapsack capacity is $W = 4$, construct the Dynamic Programming table and determine the maximum profit and the selected items.	

1.What is meant by Dynamic Programming with suitable example.

Dynamic Programming is an algorithmic technique used to solve optimization and decision problems by breaking them into smaller overlapping subproblems, solving each subproblem only once, storing the results, and reusing them to obtain the optimal solution efficiently.

Example: Fibonacci Series

The Fibonacci sequence is:

0, 1, 1, 2, 3, 5, 8, 13, ...

The recurrence relation is:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n - 1) + F(n - 2)$, for $n \geq 2$

Without DP, calculating $F(5)$ repeatedly recalculates values such as $F(3)$ and $F(2)$, making the algorithm inefficient.

2.What are the key properties for Dynamic Programming

The two key properties required to apply **Dynamic Programming** are:

1. **Optimal Substructure:** The optimal solution to a problem can be obtained by combining the optimal solutions of its subproblems.
2. **Overlapping Subproblems:** The same subproblems are solved repeatedly, so their solutions are stored and reused to avoid recomputation.

3.What are the Applications of Dynamic Programming with detail.

Applications-

Optimal binary search tree,

0/1 knapsack problem,

All pairs shortest path

problem, Traveling salesperson

problem, Reliability design.

1. Difference between Divide & Conquer (D&C) and Dynamic Programming (DP)

Divide & Conquer (D&C)	Dynamic Programming (DP)
Divides a problem into smaller independent subproblems.	Divides a problem into overlapping (interdependent) subproblems.
Follows three steps : 1. Divide 2. Conquer 3. Combine.	Follows four steps : 1. Characterize optimal solution 2. Define recurrence relation . Compute solutions (bottom-up or memoization) 4. Construct optimal solution.
Uses recursion .	Usually uses iteration (bottom-up) , though it can also use recursion with memoization.
Solves the same subproblem repeatedly if it appears multiple times.	Solves each subproblem only once and stores the result in a table (memoization/tabulation).
Uses a top-down approach.	Mainly uses a bottom-up approach (or top-down with memoization).
Subproblems are independent of each other.	Subproblems are overlapping/interdependent .
Does not store intermediate results.	Stores intermediate results to avoid repeated computation.
May perform redundant computations.	Eliminates redundant computations, making it more efficient for overlapping subproblems.
Generally simpler to design.	Requires identifying optimal substructure and overlapping subproblems.
Time complexity may be higher when subproblems overlap.	Usually reduces time complexity by reusing previously computed results.
Suitable for problems that can be split into independent parts.	Suitable for optimization problems with overlapping subproblems.
Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication.	Examples: Optimal binay search tree, 0/1 Knapsack, All-Pairs Shortest Path (Floyd–Warshall). Traveling salesperson problem

2. Define Dynamic Programming. Explain the elements of Dynamic Programming and write its general control abstraction algorithm.

Dynamic Programming is an algorithmic technique used to solve optimization and decision problems by breaking them into smaller overlapping subproblems, solving each subproblem only once, storing the results, and reusing them to obtain the optimal solution efficiently.

- **Memorization:** A **top-down** Dynamic Programming technique in which the results of solved subproblems are stored in memory and reused whenever needed, avoiding repeated computations.
- **Tabulation:** A **bottom-up** Dynamic Programming technique in which solutions to subproblems are computed iteratively and stored in a table until the final solution is obtained.

Control Abstraction for Dynamic Programing

1. Principle of Optimality :-An optimal sequence of decisions has the property that whatever the initial state and decisions are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision,
2. Optimal substructure: For the optimal solution of the problem, a solution of sub problem also must be optimal. Dynamic programming builds the optimal solution of the bigger problem using the solution of smaller sub problems. Hence we should consider only those sub problems which have an optimal solution.
3. Overlapping subproblems: When the big problem is divided into small problems, it may create exponential subproblems.

Dynamic Programming algorithm is designed using the following four steps:-

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from the computed information.
 - Dynamic programming (DP) splits the large problem at every possible point. When the problem becomes sufficiently small, DP solves it.
 - Dynamic programming is bottom up approach, it finds the solution of the smallest problem and constructs the solution of the larger problem from already solved smaller problems.
 - To avoid recomputation of the same problem, DP saves the result of sub problems into the table. When next time same problem encounters, the answer is retrieved from the table by lookup procedure.

```

Algorithm DYNAMIC_PROGRAMMING (P)

if solved(P) then
    return lookup(P)
else
    Ans  $\leftarrow$  SOLVE(P)
    store (P, Ans)
end

FUNCTION SOLVE(P)
    if sufficiently small(P) then
        solution(P)      // Find solution for sufficiently small problem
    else
        Divide P into smaller sub problems P1, P2, ..., Pn
        Ans1  $\leftarrow$  DYNAMIC_PROGRAMMIN(P1)
        Ans2  $\leftarrow$  DYNAMIC_PROGRAMMIN(P2)
        .
        .
        Ansn  $\leftarrow$  DYNAMIC_PROGRAMMIN(Pn)
        return (combine(Ans1, Ans2 ..., Ansn))
    end
end

```

Control abstraction algorithm for dynamic programming

3. Consider a 0/1 Knapsack Problem with $n=4$ items having weights $w_1=2$, $w_2=3$, $w_3=4$, and $w_4=5$ and corresponding profits $p_1=3$, $p_2=4$, $p_3=5$, and $p_4=6$. If the knapsack capacity is $C=5$, construct the dynamic programming table and determine the maximum profit and the selected items.

Given:

Item (i)	Profit (p_i)	Weight (w_i)
1	3	2
2	4	3
3	5	4
4	6	5

Given

- Number of items, $n = 4$
- Capacity, $M = W = 5$

Step 1: Apply the Formula and Build the Sequence of Decisions

Initially,

$$S^0 = \{(0,0)\}$$

where each pair represents **(Profit, Weight)**.

For i = 0 (Item 1)

$$S^0 = \{(3,2)\}$$

1

Merge

$$S^1 = S^0 \cup S^0$$

$$S^1 = \{(0,0), (3,2)\}$$

For i = 1 (Item 2)

Current

$$S^1 = \{(0,0), (3,2)\}$$

Generate

$$S^1 = \{(4,3), (7,5)\}$$

1

Merge

$$S^2 = S^1 \cup S^1$$

$$S^2 = \{(0,0), (3,2), (4,3), (7,5)\}$$

(No redundant pairs.)

For i = 2 (Item 3)

Current

$$S^2 = \{(0,0), (3,2), (4,3), (7,5)\}$$

Generate

$$S^2 = \{(5,4), (8,6), (9,7), (12,9)\} \quad 1$$

Since capacity $M = 5$,

Remove

$$(8,6), (9,7), (12,9)$$

Remaining

$$S^2 = \{(5,4)\} \quad 1$$

Merge

$$S^3 = S^2 \cup S^2$$

$$S^3 = \{(0,0), (3,2), (4,3), (5,4), (7,5)\}$$

For i = 3 (Item 4)

Current

$$S^3 = \{(0,0), (3,2), (4,3), (5,4), (7,5)\}$$

Generate

$$S^3 = \{(6,5), (9,7), (10,8), (11,9), (13,10)\} \quad 1$$

Remove pairs whose weight exceeds 5.

Remaining

$$S^3 = \{(6,5)\} \quad 1$$

Merge

$$S^4 = S^3 \cup S^3$$

Since both **(6,5)** and **(7,5)** have the same weight, keep the one with **maximum profit**.

Therefore,

$$S^4 = \{(0,0), (3,2), (4,3), (5,4), (7,5)\}$$

Step 2: Find the Optimal Solution

The maximum feasible pair is

$$(7,5)$$

Hence,

- Maximum Profit = 7
- Total Weight = 5

Step 3: Trace Back the Solution

The optimal pair

$$(7,5) \in S^4$$

Since

$$(7,5) \in S^3$$

Item 4 is **not selected**.

$$\boxed{x_4 = 0}$$

Now,

$$(7,5) \in S^3$$

Subtract Item 3:

$$(7 - 5, 5 - 4) = (2, 1)$$

This pair is **not present** in S^2 .

Hence,

$$\boxed{x_3 = 0}$$

Now,

$$(7,5) \in S^2$$

Subtract Item 2:

$$(7 - 4, 5 - 3) = (3, 2)$$

Since

$$(3, 2) \in S^1, x_2 = 1$$

Now,

$$(3, 2) \in S^1$$

Subtract Item 1:

$$(3 - 3, 2 - 2) = (0, 0)$$

Since

$$(0, 0) \in S^0, x_1 = 1$$

Final Solution

$$(x_1, x_2, x_3, x_4) = (1, 1, 0, 0)$$

Selected Items:

Item Profit Weight

1	3	2
2	4	3

Total Weight

$$2 + 3 = 5$$

Total Profit

$$3 + 4 = 7$$

Answer

- **Optimal State:** $(7, 5)$
- **Maximum Profit:** 7
- **Selected Items:** *Item 1 and Item 2*
- **Decision Vector:** $(1, 1, 0, 0)$